特 別 研 究 報 告 書

題　目

Training Large Language Models for Fusion Plasma Simulation Code Development

(核融合プラズマのシミュレーションコード開発に特化した大規模言語モデルの学習)

指導教員　成田絵美

京都大学工学部物理工学科
（原子核工学コース）

氏　　名　関　健太郎

提出年月　　令和　８　年　２　月

Training Large Language Models for Fusion Plasma Simulation Code Development

Kentaro Seki
Kyoto University
seki.kentaro.66s@st.kyoto-u.ac.jp

February 5, 2026

Abstract

This thesis studies code development for turbulent transport in fusion plasmas applying large language models (LLMs). Using a codebase called the **gyrokinetic Vlasov simulation code (GKV)**, we build a repair dataset by intentionally removing physics-critical lines and generating repair questions with known ground-truth patches. We implement **SEIMEI**, an agentic inference pipeline that navigates the repository and proposes fixes, and augment it with **RMSearch**, a reward-model-based reranker that selects task-relevant knowledge snippets during inference. Preference data for RMSearch is constructed from inference runs and trained using a Direct Preference Optimization (DPO)-style objective. This realizes a lightweight domain-adaptation loop centered on retrieval/reranking rather than retraining the base LLM. Experiments on 113 repair tasks (85 train / 28 test) show that RMSearch training reduces evaluation loss and improves reranking accuracy across iterations. When RMSearch is integrated into SEIMEI, repair accuracy improves by about 7% per iteration and exceeds 14% improvement by the third iteration compared to a baseline pipeline without RMSearch. These results suggest that knowledge retrieval and domain-specific training of RMSearch can yield practical gains in scientific code repair with modest computational cost.

Contents

1 Introduction **4**

1.1 Numerical simulations of fusion plasmas 4

1.2 Ultimate research goal: problem solving with LLMs 4

1.3 Bachelor thesis goal: code repair and its improvement 5

1.4 Organization of the thesis 5

2 Large Language Model **6**

2.1 Current Development on Large Language Models 6

2.1.1 Transformers 6

2.1.2 Scaling laws and compute-optimal training 6

2.1.3 InstructGPT / Reinforcement Learning from Human Feedback (RLHF) 7

2.1.4 Direct Preference Optimization (DPO) 7

2.1.5 DeepSeek-R1-style reasoning-focused post-training 7

2.1.6 Test-time compute (inference-time search) 8

2.1.7 Why RL can help, and why it can be expensive 8

2.2 SEIMEI 8

2.3 RMSearch 9

2.4 Goal of our research 10

2.5 Related Research 10

2.5.1 Retrieval-Augmented Generation (RAG) 11

2.5.2 AI agents (tool use, browsing, iterative editing) 11

2.5.3 DSPy (automatic pipeline improvement from evaluation) 11

2.5.4 Reranker models (relevance scoring) 12

3 Approach **13**

3.1 Overview 13

3.2 Dataset Creation 13

3.3 Inference Pipeline 14

3.4 Knowledge Generation 16

3.5 Knowledge Sampling for RMSearch Dataset 17

3.6 RMSearch Training 19

3.6.1 Reward Design . . . 19
3.6.2 Loss Function . . . 19
3.6.3 Batch construction (following InstructGPT's reward-model training recipe) 19
3.7 Training Iteration . . . 20
3.8 RMSearch & SEIMEI Evaluation . . . 20

4 Experiment **22**
4.1 RMSearch Evaluation . . . 22
4.2 SEIMEI Evaluation . . . 22
4.3 GPU and Cost . . . 22

5 Discussion **26**
5.1 Interpretation of Experimental Results . . . 26
5.2 Unsuccessful Attempts . . . 26
5.2.1 Monte Carlo Tree Search (MCTS) . . . 26
5.2.2 Knowledge Chunks Reward . . . 26
5.3 Possibility of Domain-Specific LLM . . . 27

6 Conclusion **28**

Acknowledgements **30**

1 Introduction

1.1 Numerical simulations of fusion plasmas

Fusion devices are large-scale, and plasma phenomena span wide spatiotemporal scales that cannot be fully measured. Numerical simulation is therefore essential for device design, operation scenarios, and understanding plasma physics. To cover wide scales and incorporate detailed physics, simulation codes are large and interdependent; advances in computing have improved reproducibility but raised the barrier to new contributors.

Modern nuclear-fusion research relies heavily on **gyrokinetic simulation**, where the six-dimensional Boltzmann equation is reduced into a more tractable form while preserving key microturbulence physics. This shift enabled first-principles turbulence studies and transport predictions that are difficult to obtain experimentally, and it is now routine to use large gyrokinetic codes for tokamak/stellarator turbulence and validation studies (e.g., GENE, GS2, GTC, GKV and related validation work). [1] [2] [3] [4]

However, **gyrokinetic simulation software is hard to extend**:

- **Large, interdependent codebases**: physics modules (e.g., collision operators, electromagnetic terms, geometry, diagnostics) are spread across many files and abstraction layers.
- **High cost per new feature**: adding a small physics change (e.g., a collision term, equilibrium interface, or diagnostic) often requires edits across data structures, solvers, I/O, tests, and documentation.
- **High onboarding cost**: new contributors must learn both the physics and the code architecture before making safe changes.

LLMs are becoming practical tools for software development and scientific workflows, especially when paired with **external knowledge, tools, and structured agent pipelines**. This motivates building an LLM system that helps fusion researchers understand unfamiliar codebases and implement changes more reliably.

1.2 Ultimate research goal: problem solving with LLMs

The long-term goal is to build an LLM inference system that improves reasoning and conversation with plasma researchers:

1. Improve reasoning by providing the system with source code and relevant papers/documents.
2. Improve conversation by capturing expert knowledge during interactions.
3. Reuse that knowledge in later sessions to solve new problems more effectively.

1.3 Bachelor thesis goal: code repair and its improvement

For this bachelor thesis, the near-term goal is to develop an LLM-based system that can repair a broken code derived from a gyrokinetic codebase. The main processes are:

- Create a dataset from a fusion simulation codebase.
- Train the system and evaluate its performance on repair tasks.
- Refine the workflow and knowledge pool based on observed failure modes.

1.4 Organization of the thesis

The paper is organized as follows:

Section 2.1 reviews LLM research in recent years. Section 2.2 explains SEIMEI, the inference pipeline used in this research. SEIMEI is a search-integrated agent pipeline designed to adapt the system to domain-specific tasks by training a search model. Section 2.3 describes RMSearch, a reward-model-based search/reranking architecture used in SEIMEI. Since these methods are key to this research, readers are encouraged to consult Sections 2.2 and 2.3 when focusing on the SEIMEI/RMSearch design.

Section 3 addresses the concrete method for creating an LLM system to solve GKV code-repair problems. Section 3.2 describes dataset construction with (problem, code, answer) tuples. Section 3.3 presents the inference pipeline for codebase analysis and patch generation. Section 3.4 explains knowledge creation for problem solving. Sections 3.5 to 3.7 describe knowledge-search training and evaluation.

Section 4 shows the detailed experimental setups and results.

Section 5 discusses the experimental results presented in the previous section and unsuccessful attempts we made through the experiments.

Finally, we conclude this thesis in section 6.

2 Large Language Model

2.1 Current Development on Large Language Models

Large language models (LLMs) have rapidly moved from a research topic to widely used tools, popularized by systems such as ChatGPT, and are now routinely applied in scientific workflows, including programming and code review. A core reason is that both natural language and source code are **sequential data**: interpreting a token often requires understanding *context* that may appear many steps earlier (e.g., variable definitions, function contracts, or algorithmic assumptions). This makes sequence modeling—capturing long-range dependencies while remaining trainable at scale—a central technical requirement for LLMs.

Below, we summarize key developments that explain why modern LLM systems can support non-trivial scientific coding tasks, and why recent work increasingly emphasizes post-training and inference-time search.

2.1.1 Transformers

Early neural sequence models often relied on **recurrent neural networks (RNNs)**, which process a sequence step-by-step, carrying a hidden state forward in time. While effective for short contexts, this design creates two practical bottlenecks for long, technical inputs:

- **Information compression:** the model must repeatedly compress earlier context into a fixed-size hidden state, which can degrade retention of details that appeared far in the past (e.g., a boundary condition defined hundreds of tokens earlier).
- **Limited parallelism:** because computation depends on the previous timestep, training is less parallelizable than modern architectures, slowing large-scale optimization.

The **Transformer** replaces recurrence with **self-attention**, allowing each position in the sequence to directly attend to other positions. This improves long-range dependency handling and enables highly parallel training, which made it feasible to scale both datasets and model sizes effectively. [5]

LLMs do not read text as characters or "words" directly. Instead, input is converted into a sequence of **tokens**, which are discrete units produced by a tokenizer. A token may be a full word (e.g., `function`), a subword piece (e.g., `func` + `tion`), punctuation or symbols (e.g., `{`, `=`, `;`, `}`), or short character fragments. Tokenization is widely used for efficiency: representing text at the character level yields very long sequences, while representing all words as distinct tokens creates an impractically large vocabulary. Subword tokenization offers a practical trade-off by learning reusable pieces from large corpora while keeping sequence lengths and vocabulary size manageable.

2.1.2 Scaling laws and compute-optimal training

Scaling-law work observed that model loss often follows **smooth power-law trends** as you increase model size, data size, and compute, meaning progress can be predicted and engineered.

[6] Later work showed that, under a fixed compute budget, many LLMs were "undertrained" on too few tokens, motivating **compute-optimal** training strategies (often summarized via the "Chinchilla" perspective). [7]

Why this matters for scientific coding: scaling is not only "bigger is better"—it also provides guidance for compute-efficient training, which becomes relevant when considering reinforcement-learning (RL)-based post-training costs.

2.1.3 InstructGPT / Reinforcement Learning from Human Feedback (RLHF)

Base LLMs are trained to predict the next token, not to follow instructions. InstructGPT-style pipelines added a practical recipe:

1. Collect **human-written demonstrations** (supervised fine-tuning).
2. Collect **human preference rankings** over model outputs.
3. Train a **reward model** to predict those preferences.
4. Optimize the policy using RL to improve the reward while limiting drift from the base model.

In InstructGPT, Proximal Policy Optimization (PPO) is applied to optimize the model outputs by improving the reward while constraining updates for stability. [8] [9] The broader "learning from preferences" setup is also well-established in RL. [10]

This line of work improved instruction-following and user preference alignment—even showing cases where a much smaller aligned model is preferred to a much larger base model. [8]

2.1.4 Direct Preference Optimization (DPO)

DPO reframes preference optimization as a supervised-style objective that can match RLHF-style goals (reward maximization with a KL-style constraint) without running full RL rollouts. Practically, that means:

- No explicit reward model is required at training time (it is "implicit").
- Training resembles supervised fine-tuning, which can simplify stability and reduce engineering overhead.

This matters for our setting because we want a training loop that is realistic for academic GPU budgets. [11]

2.1.5 DeepSeek-R1-style reasoning-focused post-training

Recent reasoning-focused model releases and reports (e.g., DeepSeek-R1) highlight renewed emphasis on preference optimization and RL as a path to stronger reasoning behavior, often combined with careful data and evaluation design. [12]

For this thesis's narrative, the key point is not that one model is best, but that the field trend increasingly treats **post-training (preferences, RL, or RL-free variants like DPO)** as a major lever for reasoning and reliability.

2.1.6 Test-time compute (inference-time search)

Even without changing model weights, you can often improve reasoning by spending more computation at inference time. Common patterns include:

- **Chain-of-thought prompting**: ask the model to write intermediate steps. [13]
- **Self-consistency**: sample multiple reasoning traces and pick the majority-consistent answer. [14]
- **Tree of Thoughts (ToT)**: explicitly search a tree of partial solutions, backtrack, and evaluate branches. [15]

A simple analogy: instead of trusting the first draft, you ask the model to generate multiple candidate "proof attempts," then select or refine the best. This "test-time search" theme is directly relevant to agentic coding pipelines, where we can generate multiple patch candidates and score them.

2.1.7 Why RL can help, and why it can be expensive

Empirically, preference/RL-style post-training often improves instruction-following and sometimes reasoning benchmark performance, but it can be resource-intensive because it requires generating many samples (rollouts) and performing multiple optimization steps (e.g., PPO). [8] This motivates exploring RL-free (or "more supervised-like") alternatives such as DPO where appropriate. [11]

2.2 SEIMEI

In this work we use **SEIMEI**, an open-source library that orchestrates LLM-based inference as a **search-integrated agent pipeline**. Conceptually, SEIMEI is designed for tasks requiring domain-specific knowledge and reasoning. SEIMEI realizes this by training a search model that integrates agents and knowledge and guides the inference trajectory. [16]

Key idea: instead of training the core inference LLM for next-token prediction, SEIMEI trains a search model to guide inference. Figure 1 shows how SEIMEI improves the inference pipeline by training the search model. This has two practical advantages:

1. Training the search model does not modify the core inference LLM, reducing the risk of destabilizing the base generator.
2. Adapting a search model to one domain typically requires much less computation than retraining a next-token-generation model.

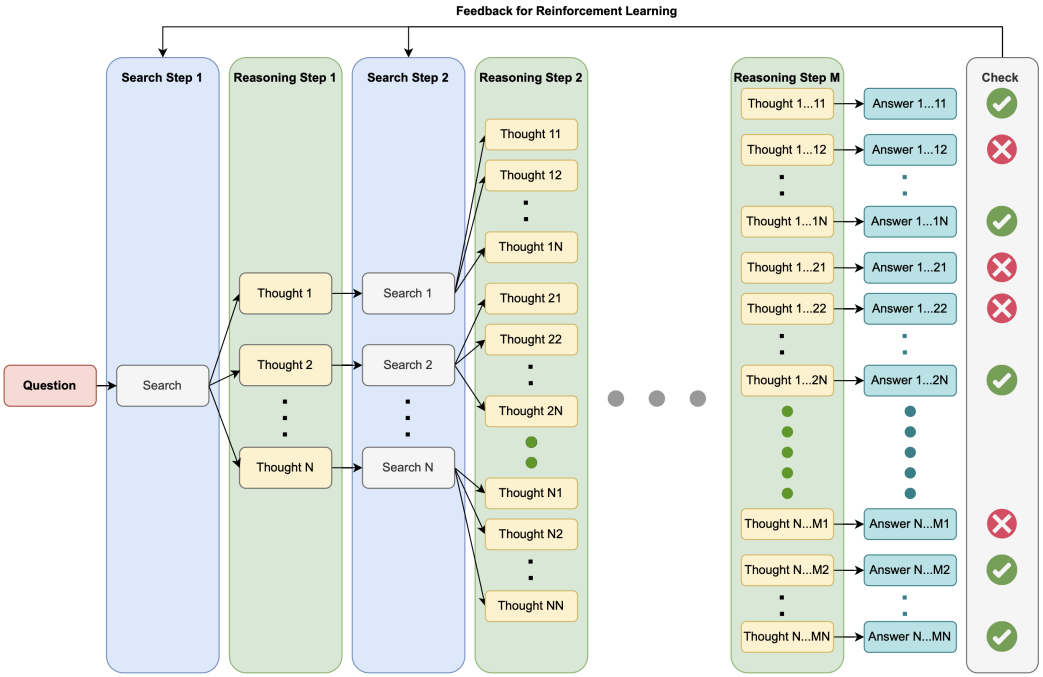

Figure 1 | SEIMEI's inference and learning pipeline. Reinforcement learning on the search model improves the reasoning path by guiding which agent/knowledge to use next.

Search has long been a key technology for knowledge access and expansion. SEIMEI extends this idea by integrating not only factual knowledge but also "how to proceed" (i.e., which agent/tool to call next), making it a candidate architecture beyond conventional workflow-agent paradigms.

2.3 RMSearch

SEIMEI's routing and knowledge selection use **RMSearch**, a reward-model-style search/reranking component. RMSearch retrieves and prioritizes helpful snippets (knowledge, prior solutions, docs, heuristics) that augment the LLM's next step. [17] The RMSearch training approach used in this thesis is also described in a KyotoAI technical blog post. [18]

Architecturally, RMSearch is closest to a **reranker**: given a query and candidate texts, it scores query–candidate relevance (often with a cross-encoder-style setup). This aligns with a long line of reranking research, including BERT-style rerankers and interaction-based models. [19] [20]

The practical difference is the retrieval pipeline. In many systems, a dense retriever first returns a top-k set (e.g., 100), and a reranker then reorders that shortlist to select the final few. RMSearch instead directly scores candidates with a reward-model-style signal, so relevance is computed with the same type of signal used for selection rather than only as a separate post-processing stage.

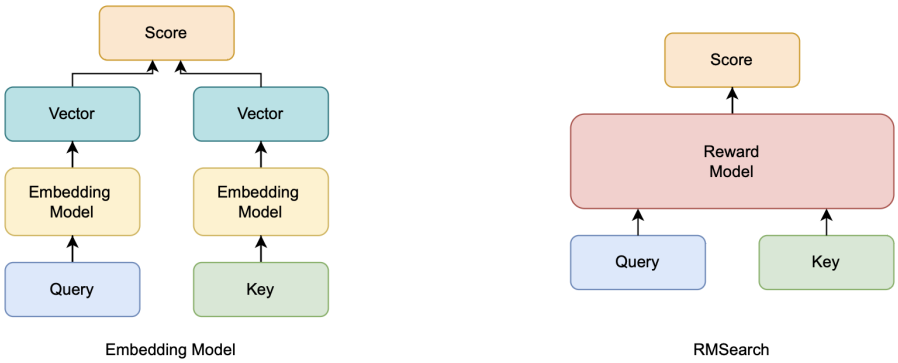

Figure 2 | Model-architecture difference between a conventional vector search model and RMSearch.

RMSearch (reranking) matters especially in domain-specific search, where generic embedding similarity can miss task-relevant signals. A dense retriever compresses text into vectors, while a reward model learns to score relevance directly from paired query–candidate text. This can make RMSearch easier to adapt to a specific domain with preference data. [17] [18]

2.4 Goal of our research

We aim to improve LLM-assisted plasma simulation code writing/editing using SEIMEI + RMSearch, with an emphasis on a gyrokinetic codebase (GKV). Our plan:

1. Build an inference system using SEIMEI.
2. Generate a dataset from a real gyrokinetic codebase.
3. Generate a **knowledge pool** by solving/repairing tasks (and extracting reusable lessons).
4. Train RMSearch (reranker-style) over that knowledge pool using preference-style data (DPO).
5. Measure whether the knowledge pool + trained RMSearch improves code-repair accuracy in the same domain.

2.5 Related Research

Below are the main research threads we build on; this section is intentionally cross-disciplinary.

2.5.1 Retrieval-Augmented Generation (RAG)

RAG methods attach an external memory (documents, snippets, code, papers) to an LLM so it can retrieve relevant context rather than relying only on parameters. The core motivation is modularity:

- You can update knowledge by updating the corpus/index, without retraining the whole model.
- You can attach provenance ("this answer used these sources").

Classic RAG work combines a neural retriever with a generator for knowledge-intensive tasks. [21] Related lines include retrieval during pretraining (REALM) [22] and retrieval-enhanced generation at very large scale (RETRO). [23] Dense retrieval also became a standard baseline for open-domain QA and retrieval pipelines. [24]

Why it matters here: codebases are "documents." A fusion code repository contains the ground truth for function contracts, data layouts, and physics assumptions. RAG-style grounding reduces hallucinated edits.

2.5.2 AI agents (tool use, browsing, iterative editing)

Modern "LLM agents" combine a language model with tools and iterative control:

- Tool use learned or prompted (e.g., Toolformer). [25]
- Modular architectures mixing LMs with specialized components (MRKL). [26]
- Reason+act prompting patterns (ReAct), useful for multi-step tasks. [27]
- Web/tool-assisted answering and reference collection (WebGPT), relevant for grounded reasoning workflows. [28]
- Software-engineering agents with repository navigation and editing interfaces (SWE-agent). [29]

Agent frameworks in practice often emphasize interfaces (file editing, running checks, browsing) rather than only better prompts—this aligns with SEIMEI's design goal of structured code repair. [29]

2.5.3 DSPy (automatic pipeline improvement from evaluation)

DSPy proposes a programming model where you declare an LLM pipeline, define a metric, and let a compiler-like optimizer improve prompts/modules using data. [30]

Connection to our goal: we also want an evaluation-driven improvement loop, but focused on (a) code repair tasks, and (b) improving retrieval/reranking (RMSearch) and knowledge pools that feed the pipeline.

2.5.4 Reranker models (relevance scoring)

Reranker models are a standard IR technique: a fast retriever gets candidates; a stronger model reorders them. BERT reranking (monoBERT) demonstrated large gains in passage ranking. [19] ColBERT provides an efficiency–quality trade-off via late interaction. [20]

Connection to our work: RMSearch plays the reranker role for "which knowledge or context should the agent use next," which becomes crucial in specialized domains like gyrokinetics.

3 Approach

In this section, we present our approach to develop a pipeline for repairing gyrokinetic code under a setting where an error is introduced into the code. After showing an overview of the workflow in Section 3.1, we describe each component of the workflow in Sections 3.2–3.8.

3.1 Overview

Our framework is composed of four main parts: (1) Dataset, (2) Inference, (3) Sampling, and (4) Training. We perform the following tasks to develop the framework:

1. **Dataset creation (Section 3.2):** We prepare a training dataset by intentionally breaking the code and creating a question that asks for a repair.
2. **Inference pipeline (Section 3.3):** For each (question, broken codebase), we run an inference pipeline that generates candidate repair patches and intermediate traces.
3. **Scoring (Section 3.6.1):** The candidate repairs are evaluated using static checks, diff-quality checks, and (when available) tests/compilation.
4. **Knowledge generation (Section 3.4):** We collect reusable "lessons" from successful (and failed) repairs for effective repair of highly specialized codes.
5. **RMSearch dataset sampling (Section 3.5):** We create preference pairs by sampling inference traces and candidate knowledge items.
6. **Train RMSearch (Sections 3.6 and 3.7):** DPO-style preference optimization is applied to train RMSearch.
7. **Evaluation (Section 3.8):** We run the inference pipeline again, now routed/augmented by the trained RMSearch.

This framework is designed to be feasible without full RLHF infrastructure, while still leveraging preference-style learning.

3.2 Dataset Creation

We use the open gyrokinetic code GKV as the source code to generate the dataset. [31] GKV is written in Fortran, and its codebase consists of many Fortran source files as mentioned in Section 1.1.

We assume an error is introduced into the code, and our framework assists with repair. A representative prompt is "repair the code so it matches the original intended physics-aware behavior." To train the model inside the framework, we prepare the training dataset using an LLM with the target repository as follows:

1. An LLM selects which line(s) to break, with the constraint that the deletion should affect core physics logic (not only comments or trivial formatting).

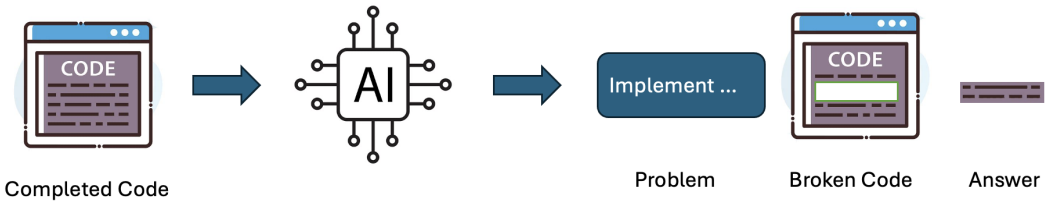

Figure 3 | Dataset generation pipeline. An LLM intentionally breaks complete code and generates problems related to repairing the code snippet.

2. The pipeline generates:
   - a patch that removes important lines,
   - a question describing the observed failure or missing behavior ("restore the missing computation / boundary condition / term") and asking the system to fix the issue.
3. Patch debugging: a generated patch is sometimes invalid when applied to the file, so an LLM iteratively fixes the patch based on error messages. If a patch still fails after this process, it is removed.

In this way, many (problem, answer) pairs are obtained to define the break-and-repair task.

Why break-and-repair is useful:

- **Correctness:** It provides a clean ground-truth patch because the correct fix is known. The system can compare the output patch with the original code.
- **Scalability:** This method only requires complete code and LLM calls, and can be scaled to other codes in plasma physics or other scientific domains.
- **Realism:** It creates physics-centered code editing problems that researchers often encounter in small additions or fixes related to plasma equations.

3.3 Inference Pipeline

We implement the inference loop using SEIMEI. [16] Figures 4 and 5 show the difference between a baseline agent loop and the SEIMEI approach.

In the baseline approach, three agents are used in analyzing GKV code:

- **code_act agent:** runs Unix commands (e.g., `cat`, `ls`, `rg`) or Python code to analyze local folders and files.
- **think agent:** reviews command results and prior analysis, then decides what to do next.

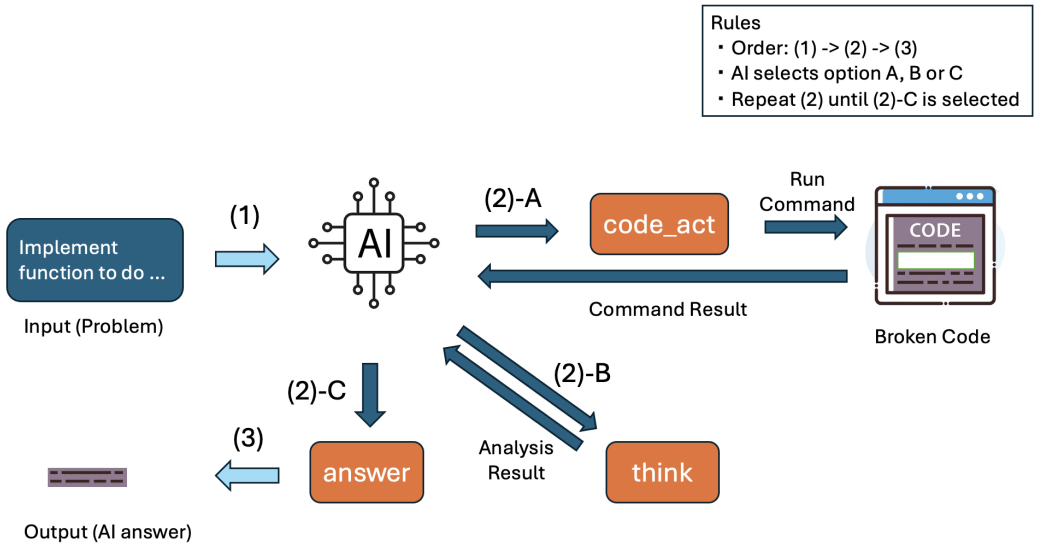

Figure 4 | Current approach to access a codebase and solve problems. A generative model selects a code_act agent to access files and a think agent to decide next steps; once sufficient information is collected, an answer agent produces the final output.

- **answer agent:** gives the final answer summarizing the analysis results.

These agents work as follows:

1. A problem is passed to a next-token generation model, which selects which agent to use in the next step.
2. Each agent produces an output. The model then conditions on the history of outputs to select the next agent. This repeats until the answer agent is selected.
3. After the answer agent is chosen, it summarizes what the agents found and outputs the result.

This aligns with patterns demonstrated in prior agent work for tool use and repository-scale editing. [27] [25] [26] [29]

Compared to these methods, SEIMEI adds **routing and augmentation**: at each step, SEIMEI can call RMSearch to retrieve domain-relevant knowledge snippets, then incorporate them into the next model call. [16] [17]

The SEIMEI loop is:

1. A problem is passed to RMSearch instead of directly to the next-token generation model.
2. RMSearch selects what knowledge should be retrieved in the current step. Each knowledge item specifies which agent should use it, and the system calls that agent.

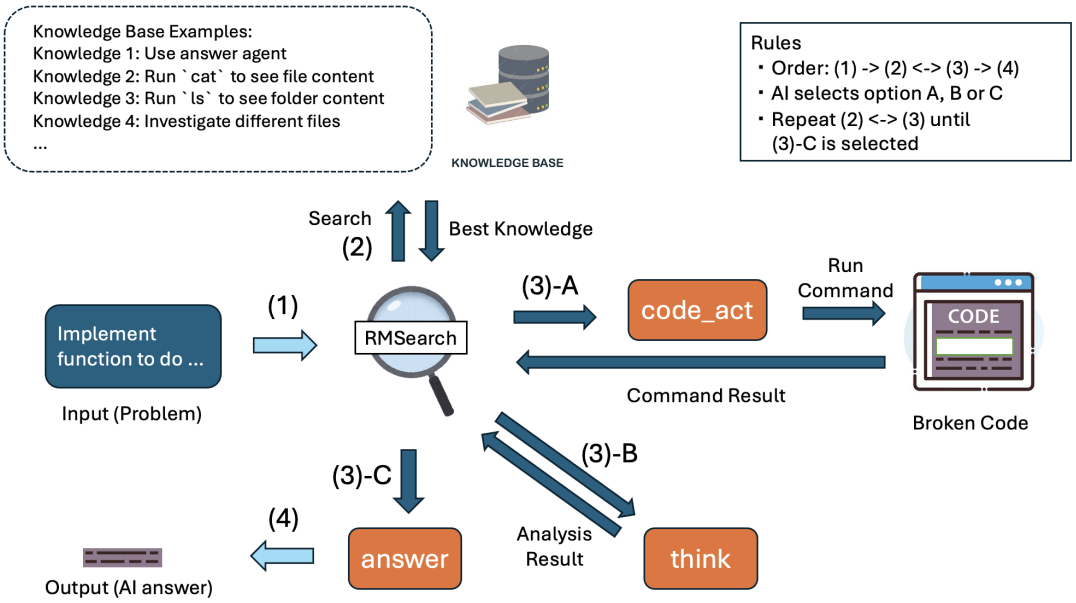

Figure 5 | SEIMEI approach to access a codebase and solve problems. Compared to the current method, SEIMEI integrates knowledge and agents through RMSearch, making the overall system depend on RMSearch-based routing.

3. Each agent uses the retrieved knowledge and produces an output. The system then returns to (2) with the history of outputs until RMSearch chooses the answer agent.

4. The answer agent summarizes the results and outputs the final response.

We use RMSearch rather than a conventional semantic-embedding retriever because RMSearch (as a reranker-style model) computes relevance more directly and can be adapted to a specific domain using preference data. [17] [18]

3.4 Knowledge Generation

The knowledge pool can contribute to effective repair of highly specialized codes like GKV. In this study, the knowledge pool is generated in two steps:

1. **Manual knowledge:** Domain experts (or careful readers) write concise instructions about the task, such as:

   - "Run the answer agent because enough agent outputs are obtained."
   - "Run `ls` to see what is in the current folder."
   - "Run `cat` to see what is inside a file."
   - "Run `rg` to search keywords over a folder."
   - "Change the strategy to file-comparison-centered analysis."

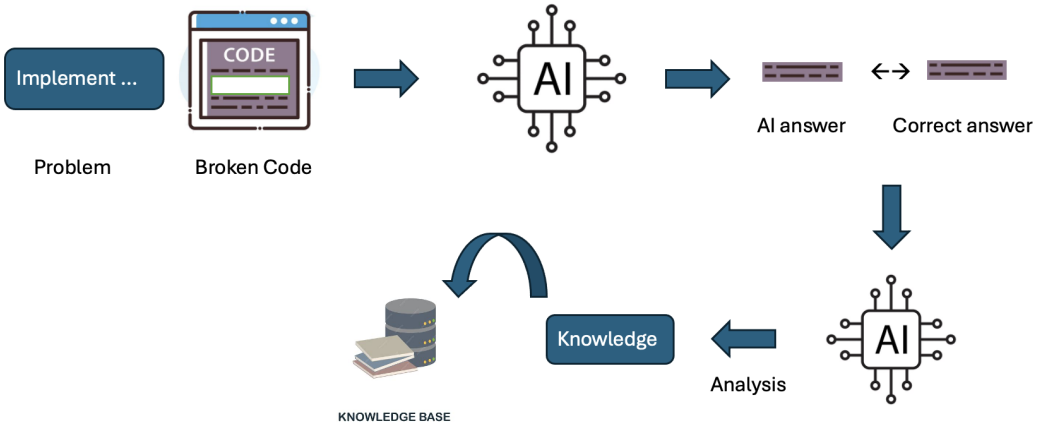

Figure 6 | Knowledge update pipeline from correct answer.
Solved repair tasks are converted into reusable knowledge that improves future code-repair performance. A problem and broken code snippet are given to an AI agent, which produces an answer that is compared with the correct solution. The comparison is distilled into structured knowledge, which is stored in a knowledge base and later retrieved to guide subsequent repairs.

Writing this requires careful analysis of reasoning steps and AI answers.

2. **Automatic knowledge update (from repairs):** Compare the agent's output patch to the expected fix and extract reusable statements such as:
   - "When you run `cat`, run `ls` first to confirm the file path in the current folder."
   - "You often cause errors with `rg`; follow the format: ___."
   - "When modifying the gyro-averaged potential term, also update normalization in _____."
   - "This file assumes flux-tube geometry; boundary conditions are enforced in _____."

The goal is to turn one solved instance into a hint that helps future instances.

We keep both because automatic knowledge can scale, while manual knowledge can be higher precision.

3.5 Knowledge Sampling for RMSearch Dataset

RMSearch training requires a preference-style dataset. We construct it from pipeline runs by creating comparisons such as:

- **Knowledge preference:** given a query ("fix this missing physics term") and previous

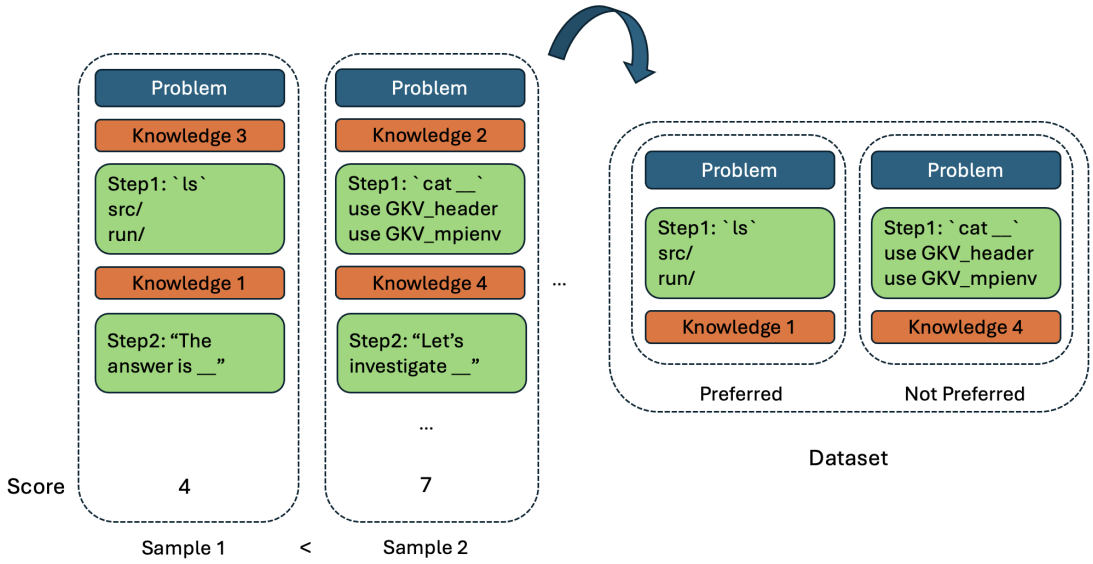

Figure 7 | Preference-style dataset creation for DPO training.

agent outputs, compare two candidate knowledge snippets A vs B based on whether using them leads to a higher-scoring answer.

This matches the general "learning from preferences" paradigm used in RLHF, but we apply it to retrieval/reranking and agent routing rather than directly to the generator model. [32]

In this method, knowledge sampling and preference assignment are key. The sampling procedure used in this experiment is:

- **Inference:** Knowledge sampling starts from running SEIMEI on problems. In each step, SEIMEI selects a knowledge text that is expected to be most useful for the next action. There are two knowledge-selection modes: an "llm" mode where a next-token model selects knowledge, and an "rm" mode where a reward model selects knowledge. Each inference produces a trace of the form (problem, number_of_steps × (knowledge, agent_output)).
- **Sampling:** To construct a dataset for RMSearch, we sample a fixed number of candidate knowledge items per step (according to the sampling configuration used in the experiment).
- **Scoring knowledge:** After the pipeline produces an answer, we score the answer using the reward design described in Section 3.6.1.
- **Converting to a preference dataset:** From scores, we build (chosen set, rejected set) pairs. Each set includes the message history up to the step that will be augmented by the knowledge. A threshold removes trivial score differences.

3.6 RMSearch Training

3.6.1 Reward Design

A key difference from a standard RAG pipeline is that RMSearch is trained on a reward derived from the quality of the final repair. The reward is computed by comparing the model-generated patch with the ground-truth patch (the original code before deletion). We use a 0–10 score with the following components. Table 1 summarizes the rubric and the purpose of each component.

- +2 for modifying the correct file.
- +2 for modifying the correct location in that file.
- +3 for restoring the same functional code as the deleted portion.
- +3 for how directly the retrieved knowledge contributes to the reasoning steps (heuristically: +1 if the knowledge identifies the correct file, +1 if it identifies the correct code snippet, +1 if it materially improves the reasoning path).

This design keeps the reward tied to correctness while also encouraging RMSearch to select knowledge that is actually used by the agent to reach the fix.

3.6.2 Loss Function

RMSearch is trained with DPO-style preference optimization, treating RMSearch as a model that should assign higher relevance scores to preferred items (knowledge snippets or patches). [11]

Concretely, RMSearch implements a scoring function that maps a query–key pair to a scalar score, i.e., $s_\theta(q, k) \to r$. For a preference triple (q, k^+, k^-), we use the DPO loss in a standard reference-model form:

$$\mathcal{L}_{\text{DPO}} = -\log \sigma\Big(\beta \Big[\big(s_\theta(q, k^+) - s_{\text{ref}}(q, k^+)\big) - \big(s_\theta(q, k^-) - s_{\text{ref}}(q, k^-)\big)\Big]\Big),$$

where q is the query, k^+ is the preferred key (knowledge or candidate patch), k^- is the less-preferred key, s_θ is the RMSearch scoring function, s_{ref} is a fixed reference scorer used to keep updates conservative, β controls the sharpness of the preference margin, and σ is the logistic function. This makes the connection to RMSearch explicit: training adjusts the query–key scores so that preferred knowledge/patches receive higher scores, which directly improves reranking behavior at inference time.

3.6.3 Batch construction (following InstructGPT's reward-model training recipe)

A practical issue in preference training is that many pairwise comparisons derived from the same query are highly correlated. In the InstructGPT reward-modeling pipeline, labelers rank K candidate completions per prompt (with K between 4 and 9), yielding up to $\binom{K}{2}$ pairwise

comparisons; the authors found that naively shuffling these comparisons and training on them as independent datapoints caused rapid overfitting. Instead, they treat all $\binom{K}{2}$ comparisons from a single prompt as one batch element, which is both (i) more compute-efficient (one forward pass per candidate rather than $\binom{K}{2}$ passes) and (ii) more stable (reduced overfitting, improved validation loss/accuracy). [8]

We adopt the same batching principle for RMSearch: each minibatch contains B distinct queries, and for each query we score K candidate keys and apply the pairwise preference loss over the within-query comparison set, rather than mixing comparisons across queries indiscriminately. This keeps updates aligned with the within-query reranking structure that RMSearch must solve at inference time.

3.7 Training Iteration

RMSearch training and SEIMEI inference form a bootstrapping loop. RMSearch is trained on data produced by SEIMEI runs, and the improved RMSearch then changes which knowledge is retrieved in the next round of SEIMEI inference. Concretely, each iteration follows:

1. Run SEIMEI with the current RMSearch (or a baseline retriever in iteration 0) to generate repairs and collect logs.
2. Build preference data from these runs (knowledge chunks or candidate patches) using the reward defined in Section 3.6.1.
3. Train RMSearch with the DPO objective.
4. Use the updated RMSearch for the next iteration of inference and data collection.

This loop is intended to steadily improve retrieval quality, which in turn improves downstream repair accuracy without retraining the base LLM.

3.8 RMSearch & SEIMEI Evaluation

We use two metrics to evaluate RMSearch:

- **Evaluation loss:** the DPO loss defined in Section 3.6.2, which should decrease as the model learns to prefer better knowledge.
- **Accuracy:** the percentage of cases where RMSearch assigns the highest score to the preferred key for a given query.

For SEIMEI, we evaluate the inference pipeline using the same reward function defined in Section 3.6.1. We report accuracy based on whether the generated patch is judged correct under this scoring rule, and we also track reward trends across iterations.

Table 1: **Reward design for RMSearch.** Scoring components and their maximum contributions to the 0–10 reward.

Component	Max Score	Evaluation Criterion	Purpose
Correct File Modification	+2	The generated patch edits the same file as the ground-truth patch.	Ensures the model navigates to the right source file.
Correct Location in File	+2	The edit occurs at the correct line or code region.	Rewards precise localization of the defect.
Functional Code Restoration	+3	The restored code matches the functionality of the deleted/original code, even if syntax differs slightly.	Encourages semantic correctness over surface similarity.
Knowledge Contribution — File Identification	+1	Retrieved knowledge helps identify the correct file.	Incentivizes useful retrieval for navigation.
Knowledge Contribution — Snippet Identification	+1	Retrieved knowledge points to the correct code segment.	Rewards retrieval that narrows the search space.
Knowledge Contribution — Reasoning Improvement	+1	Retrieved knowledge materially improves the reasoning path toward the fix.	Promotes retrieval that actively supports decision making.
Total Possible Score	**10**	Sum of all components.	Balances correctness and effective knowledge usage.

4 Experiment

We evaluate the dataset generated by the method in Section 3.2 using the GKV codebase. The dataset contains 113 repair tasks, split into 85 training problems and 28 test problems.

Models

- Inference model: `openai/gpt-oss-20b`
- Reward model / RMSearch backbone: `Skywork/Skywork-Reward-V2-Qwen3-4B` (referred to as `qwen4b-reward`)

Key hyperparameters

- Evaluation: Seven sampling runs per problem.
- Training: preference threshold = 0.5.
- Iteration 1: knowledge sampling model = `gpt-oss-20b`, distribution decay = 0.8, random sampling rate = 0.1, sampling number = 14.
- Iteration 2–3: knowledge sampling model = RMSearch trained in the previous iteration, distribution decay = 0.5, random sampling rate = 0.1, sampling number = 14.

4.1 RMSearch Evaluation

Figure 8 shows the evaluation log for training RMSearch. The evaluation loss decreases steadily across iterations, while accuracy increases, indicating that the RMSearch model becomes better at selecting the preferred knowledge/key for each query. Iteration 1 shows a different accuracy trajectory from iterations 2 and 3, which is expected because iteration 1 uses the base LLM for knowledge sampling, while later iterations use the trained RMSearch from the previous round.

4.2 SEIMEI Evaluation

Figure 9 shows the evaluation result of testing SEIMEI pipeline without RMSearch and with trained RMSearch of iteration 1 and 2. Compared with the base pipeline, the SEIMEI pipeline augmented by RMSearch shows consistent gains. Accuracy improves by roughly 7% in each iteration, and by the second iteration the cumulative improvement exceeds 14%. This suggests that better retrieval and reranking translate into tangible improvements in end-to-end repair performance.

4.3 GPU and Cost

We used the following hardware:

- `gpt-oss-20b` inference: RTX A5000 (24 GB VRAM)

- `qwen4b-reward` search: RTX A5000 (24 GB VRAM)
- `qwen4b-reward` training: A40 (48 GB VRAM)

Per iteration, the runtime and cost were approximately:

- ~6 hours for knowledge sampling (2× RTX A5000)
- ~6 hours for RMSearch training (1× A40)

The total cost was about $6 per iteration using a Runpod rental server.

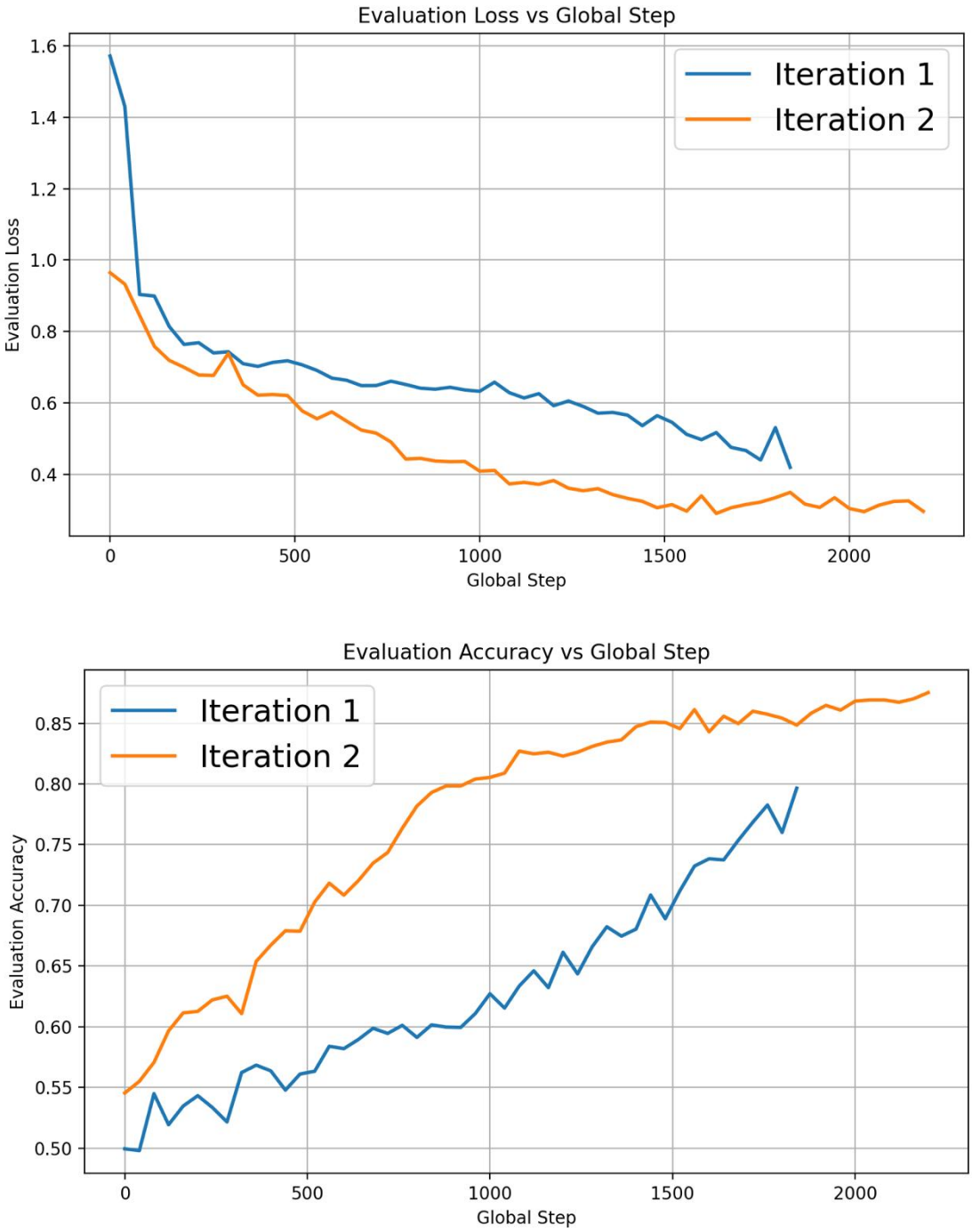

Figure 8 | RMSearch training evaluation log across iterations.

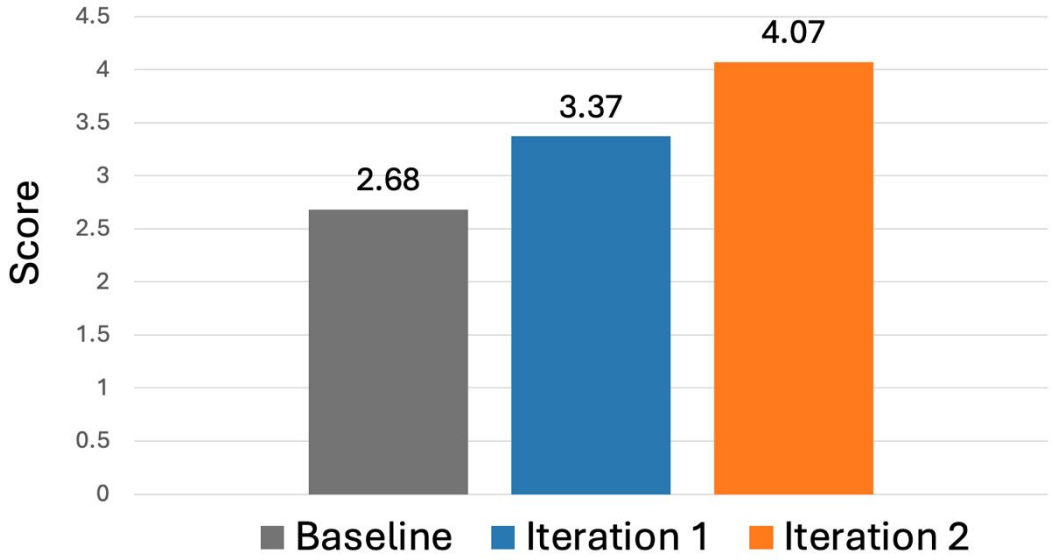

Figure 9 | SEIMEI evaluation across iterations. "Base" denotes the baseline inference pipeline in Section 3.3 without RMSearch-based routing.

5 Discussion

In this section, we discuss the experimental results presented in section 4. In section 5.1, we explain how to interpret the results and give insights into the ability of the framework developed in this study. As we have arrived at the final framework by trial and error, we describe several modifications in section 5.2. Also, the advantages of the final framework will be presented in section 5.3.

5.1 Interpretation of Experimental Results

The results are promising because they show measurable improvement from a single domain-specific codebase without retraining the base LLM. The gains suggest that RMSearch does more than refine existing model outputs: by selecting task-relevant knowledge, it injects information the base model would not reliably recall on its own.

The iterative improvement across three rounds indicates that the bootstrapping loop is effective: better reranking produces better inference traces, which in turn yields higher-quality preference data for the next round. Importantly, this improvement is achieved with relatively low compute compared to full language-model training, because RMSearch training focuses on reranking rather than next-token generation.

At the same time, these experiments are limited to one codebase and a break-and-repair task formulation. The results should therefore be interpreted as evidence that the approach is viable and cost-effective, but not yet as proof of broad generalization.

5.2 Unsuccessful Attempts

When generating the dataset and sampling knowledge, we also encountered failures and setbacks along the way. We share these failure experiences here to provide insight; however, this does not imply that these approaches cannot develop effective search models.

5.2.1 Monte Carlo Tree Search (MCTS)

In early experiments, we integrated MCTS to guide step-by-step knowledge selection. The approach did not work well because the reward signal at each step was too noisy; the search tended to optimize short-term signals without improving the final repair quality. As a result, the system did not evolve meaningfully.

5.2.2 Knowledge Chunks Reward

To address the failure of step-wise rewards, we introduced "knowledge chunks," which are bundles of knowledge spanning multiple steps and generated by an LLM from a base inference. Figure 10 shows the workflow to make preference-styled dataset from rerunning the inference with knowledge chunks. Scoring these chunks produced a more stable reward signal than step-level scoring. However, this method reduces flexibility: the knowledge is fixed for multiple

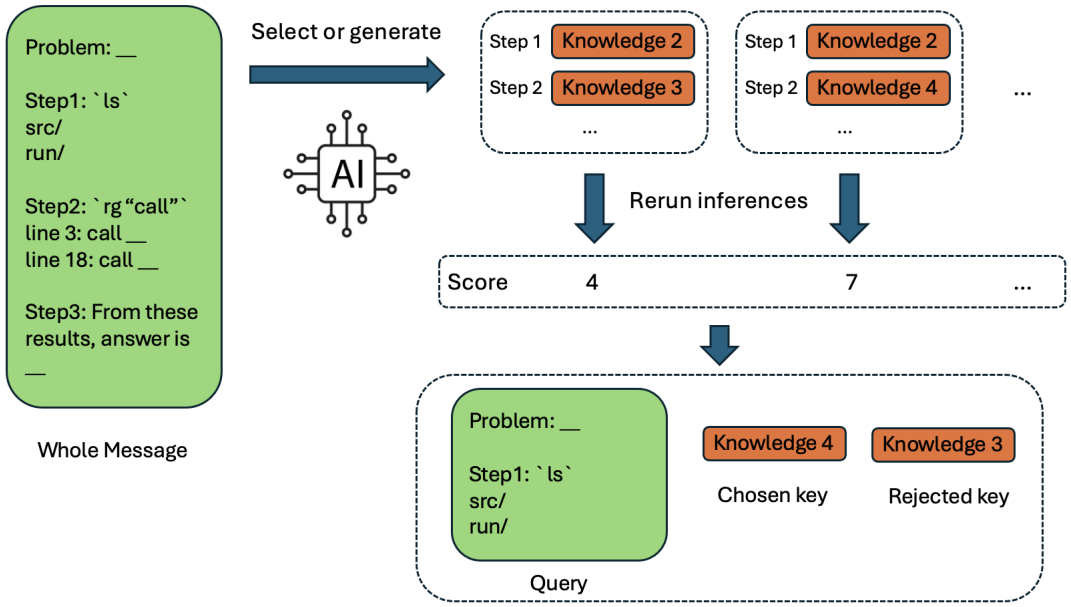

Figure 10 | Knowledge chunk type sampling method.

steps and can conflict with the model's evolving needs during inference, which suppressed adaptive sampling and limited overall gains.

5.3 Possibility of Domain-Specific LLM

Although we do not retrain the base LLM in this study, the results suggest a pathway toward near real-time learning in a domain. The key is to treat knowledge retrieval and reranking as the adaptive component: new repairs, discussions, or expert notes can be added to the knowledge pool immediately, and RMSearch can be updated with lightweight preference training. Because this training operates on a small reranker rather than the full LLM, it can be performed frequently and at low cost, enabling rapid adaptation to new code changes or newly discovered best practices.

In practice, a real-time learning loop would consist of (1) logging successful and failed repairs, (2) extracting reusable knowledge, and (3) periodically or continuously updating RMSearch. The base LLM remains stable, while the search model and knowledge pool evolve with the repository. This separation provides a form of real-time learning without the risks of catastrophic forgetting or the high compute cost of full model fine-tuning. Future work should test continuous updates and establish safeguards (e.g., validation gates) to prevent noisy or incorrect knowledge from degrading performance.

6 Conclusion

This thesis proposed and evaluated a practical framework for **domain-adapting an LLM-based coding assistant to a single, specialized scientific codebase** without retraining the base generator model. Focusing on turbulent transport simulation for fusion plasmas, we used the open-source **gyrokinetic Vlasov code GKV** as a realistic target where correctness is physics-sensitive and code understanding requires repository-scale context.

Our first contribution is the construction of a **break-and-repair dataset** tailored to scientific computing. By intentionally removing physics-critical lines and pairing each deletion with a ground-truth patch, we obtained training and test problems with an unambiguous notion of correctness. This dataset design is especially valuable for scientific software, where evaluation is often difficult: by anchoring tasks to known patches, we can measure repair success beyond surface-level plausibility and directly connect model behavior to code semantics.

Our second contribution is an **agentic inference pipeline** implemented with SEIMEI, which structures repository exploration (file listing, reading, keyword search, and step planning) and produces candidate patches with intermediate traces. Rather than relying purely on prompt engineering, SEIMEI treats code repair as a multi-step process that can be logged, scored, and improved, making it suitable for iterative development and analysis.

The core contribution of this thesis is demonstrating that **retrieval/routing can be the main learnable component** for domain adaptation. We constructed a **knowledge pool** from both manual heuristics and automatically distilled lessons from solved repairs, and trained **RMSearch** as a reranker-style model using **DPO-style preference optimization**. The reward design explicitly ties training signals to end-to-end repair quality (correct file, correct location, functional restoration, and measured knowledge contribution), and the batching strategy follows the practical insight from reward-model training that within-query comparisons should be treated as a structured unit.

Experiments on 113 repair tasks show consistent trends: RMSearch evaluation loss decreases and accuracy increases across iterations, and when RMSearch is integrated into SEIMEI, **end-to-end repair accuracy improves by roughly 7% per iteration**, exceeding **14% cumulative improvement by the third iteration** compared to a baseline pipeline without RMSearch-based routing. Importantly, these gains were achieved at **modest computational cost** (on the order of a few GPU-hours per stage and approximately $6 per iteration in our Runpod setting), supporting the thesis's central claim that *lightweight domain adaptation via reranking is feasible for academic-scale resources.*

There are clear limitations. The current evaluation is restricted to a single codebase and a synthetic break-and-repair formulation, and the reward relies on comparison to ground-truth patches rather than fully physics-based runtime validation. In addition, knowledge extraction quality can vary: noisy or overly general "lessons" may reduce usefulness as the knowledge pool scales, suggesting the need for stronger filtering and validation gates.

Future work naturally extends along three directions. First, we will evaluate generality by expanding to additional gyrokinetic and plasma-physics repositories and by incorporating more realistic defect sources (e.g., refactoring mistakes, boundary-condition regressions, and diagnostic inconsistencies). Second, we will strengthen correctness checks using compilation/tests and,

where feasible, physics-facing validation signals (e.g., conservation properties or regression metrics on representative runs). Third, we will improve the knowledge lifecycle: automated distillation, deduplication, and confidence scoring, combined with periodic RMSearch updates, would move the system toward a **continuous, low-cost learning loop** where the base LLM remains stable while the search model and external memory evolve with the repository.

Overall, this thesis provides evidence that **domain-specific retrieval and preference-trained reranking** can materially improve scientific code repair, offering a concrete path toward reliable LLM assistance for complex simulation software under realistic compute constraints.

Acknowledgements

I would like to express my sincere gratitude to **Ms. Narita** for her guidance and support throughout this project. Her careful feedback helped me refine both the experimental design and the writing, and her encouragement made it possible to continue iterating through failures and improvements until the system became measurable and stable.

I am also deeply thankful to my colleagues at **KyotoAI Inc.**—**Saxena Prakhar**, **Juan Pablo**, **Mingkwan Rattanasiwamok**, and **Roshia**—for sharing foundational ideas and practical insights on RMSearch and for the many discussions that clarified what matters in real engineering workflows: reproducibility, careful logging, and evaluation that connects model behavior to outcomes. Their day-to-day support and willingness to answer small questions made a substantial difference in my progress.

I would like to acknowledge the **GKV developers** and the broader fusion and gyrokinetics research community for producing and maintaining high-quality scientific software and publications. Open repositories and well-documented methods are essential for making research in this area reproducible and extensible, and this thesis was only possible because such resources are publicly available.

I also appreciate the creators and maintainers of the open-source tools and libraries that supported this work, as well as the infrastructure providers that enabled practical experimentation under limited time and hardware constraints, including the GPU resources used for inference and training in our iterative runs.

Finally, I would like to thank my family for their constant support. Their patience and encouragement sustained me through long debugging sessions, repeated experimental iterations, and the writing process. This work is the result of that support as much as it is of the experiments described in the thesis.

References

[1] Görler et al., *The global version of the gyrokinetic turbulence code GENE*, 2011.

[2] Dorland et al., *Gyrokinetic Simulations of Tokamak Microturbulence* (GS2-related overview), 2000.

[3] Holod et al., *Gyrokinetic particle simulations… (GTC-related)*, 2008.

[4] Nakata et al., *Validation studies of gyrokinetic ITG and TEM turbulence…*, *Nucl. Fusion* 2016.

[5] Vaswani et al., *Attention Is All You Need*, NeurIPS 2017. arXiv:1706.03762.

[6] Kaplan et al., *Scaling Laws for Neural Language Models*, 2020. arXiv:2001.08361.

[7] Hoffmann et al., *Training Compute-Optimal Large Language Models*, NeurIPS 2022. arXiv:2203.15556.

[8] Ouyang et al., *Training language models to follow instructions with human feedback (InstructGPT)*, NeurIPS 2022. arXiv:2203.02155.

[9] Schulman et al., *Proximal Policy Optimization Algorithms*, 2017. arXiv:1707.06347.

[10] Christiano et al., *Deep Reinforcement Learning from Human Preferences*, NeurIPS 2017. arXiv:1706.03741.

[11] Rafailov et al., *Direct Preference Optimization (DPO)*, 2023. arXiv:2305.18290.

[12] DeepSeek-AI, *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*, 2025. arXiv:2501.12948.

[13] Wei et al., *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*, 2022. OpenReview.

[14] Wang et al., *Self-Consistency Improves Chain of Thought Reasoning in Language Models*, 2022. arXiv:2203.11171.

[15] Yao et al., *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*, NeurIPS 2023. OpenReview.

[16] KyotoAI, *SEIMEI (repository)*. GitHub repository.

[17] KyotoAI, *RMSearch (repository)*. GitHub repository.

[18] KyotoAI, *RMSearch: Training Reward Models for Intelligent Search Using Advanced DPO Batching*, Medium, Feb 2026. https://medium.com/@kyotoaiblog/rmsearch-training-reward-models-for-intelligent-search-using-advanced-dpo-batching-ee3a4c9ccabc

[19] Nogueira & Cho, *Passage Re-ranking with BERT (monoBERT)*, 2019. arXiv:1901.04085.

[20] Khattab & Zaharia, *ColBERT*, 2020. arXiv:2004.12832.

[21] Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, NeurIPS 2020. arXiv:2005.11401.

[22] Guu et al., *REALM: Retrieval-Augmented Language Model Pre-Training*, 2020. arXiv:2002.08909.

[23] Borgeaud et al., *Improving language models by retrieving from trillions of tokens (RETRO)*, ICML 2022. arXiv:2112.04426.

[24] Karpukhin et al., *Dense Passage Retrieval for Open-Domain QA*, EMNLP 2020. arXiv:2004.04906.

[25] Schick et al., *Toolformer: Language Models Can Teach Themselves to Use Tools*, 2023. arXiv:2302.04761.

[26] Karpas et al., *MRKL Systems*, 2022. arXiv:2205.00445.

[27] Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, ICLR 2023.

[28] Nakano et al., *WebGPT: Browser-assisted question-answering with human feedback*, 2021. arXiv:2112.09332.

[29] Yang et al., *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*, NeurIPS 2024. arXiv:2405.15793.

[30] Khattab et al., *DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines*, 2023. arXiv:2310.03714.

[31] GKV-developers, *gkvp: GyroKinetic Vlasov simulation code (repository)*. GitHub repository.

[32] Ziegler et al., *Fine-Tuning Language Models from Human Preferences*, 2019. arXiv:1909.08593.

[33] Brown et al., *Language Models are Few-Shot Learners*, NeurIPS 2020. arXiv:2005.14165.

[34] Stiennon et al., *Learning to summarize from human feedback*, NeurIPS 2020. arXiv:2009.01325.

[35] Chen et al., *Evaluating Large Language Models Trained on Code (Codex)*, 2021. arXiv:2107.03374.

[36] OpenHands, *OpenHands (coding agent platform, repository/site)*. GitHub repository.